Roofline-Guided Characterization (RGD) of Attention Decode on Blackwell: From Per-CTA-Bound to Work-Starvation Across MHA, GQA, and MLA

Kien Pham
Tufts University

Abstract—Attention decode is the latency-critical phase of large language model (LLM) serving, yet no published work provides a prediction-vs-measured roofline characterization on NVIDIA's latest datacenter GPU (B300 / sm_103) across attention types and precisions. I present Roofline-Guided Decode (RGD): twelve attention kernels of increasing sophistication (naive, tiled, online-softmax, fused, tensor-core WMMA, split-KV, paged, GQA-packed, FP8, NVFP4, MLA, and tcgen05), each paired with a pre-registered two-layer roofline prediction (pure roofline plus a per-CTA-corrected layer) and evaluated against measured results on Tesla T4, A100, B200, and B300. My central finding is that decode's apparent per-CTA throughput ceiling is work-starvation: the right engine (tcgen05 tensor cores) converts work into throughput (0.75 → 1785 TFLOP/s on B300), but only in the high-throughput serving regime (large batch × long context). At single-stream, moderate-context decode, even NVIDIA's production kernel runs at 1–2% of the FP8 peak. Across the seven decode steps the per-CTA-corrected layer matched the measured limiter every time, while pure roofline predicted the wrong regime at every single-stream step. As a pre-registered boundary test, I confirm on B300 that native NVFP4 compute does not accelerate MLA decode: both FP4 and FP8 GEMMs remain HBM-bound at the M=128 decode shape, reaching at most 5.8% and 21.7% of their tensor-core peaks. I complement, not beat, production kernels: the contribution is the open, prediction-vs-measured characterization, the two-layer model, and the boundary results.

Index Terms—roofline model, attention, LLM inference, decode, Blackwell, GPU performance, MLA, GQA, FP8, NVFP4, quantization

I. Introduction

Large language model (LLM) serving latency is dominated by attention decode: after the prompt is processed, tokens are generated one at a time, and each new token attends over the entire growing key–value (KV) cache. Because a single query row (N_q=1) reads a large KV cache to produce one output, decode is the archetypal memory-bound, low-arithmetic-intensity workload, and it is where production systems spend most of their wall-clock time. Highly tuned kernels such as FlashInfer [1], FlashMLA [2], and the FlashAttention lineage [3]–[6] are deployed on NVIDIA's Blackwell GPUs, but the open literature contains no prediction-vs-measured roofline characterization of attention decode on the latest datacenter part (B300 / GB300, sm_103). Production write-ups report headline throughput; they do not publish, per kernel, what the roofline predicted, what was measured, and why the two differ.

a) The empty cell: Figure 1 maps the landscape. FlashAttention-4 [6] targets and benchmarks B200 (sm_100) for BF16 prefill/training; its kernel is deployed but never characterized on sm_103. The closest prior open roofline of MLA-style decode is Grouped-Latent Attention (GLA) on H100 [7]; my delta is exactly sm_103 plus KV-quantization. SnapMLA [8] studies FP8 MLA decode on Hopper without a Blackwell roofline; SageAttention3 [9] and Attn-QAT [10] push FP4 attention but for prefill. Microbenchmark-driven analytical modeling [11] establishes why naive roofline fails on modern GPUs and owns the method; I cite and apply it. The cell for an open, prediction-vs-measured decode roofline on sm_103, across attention types and precisions, is empty. I fill it with Roofline-Guided Decode (RGD), the study this paper presents.

b) Contributions:

1) The first open, prediction-vs-measured roofline characterization of attention decode on B300 / sm_103, spanning three attention types (MHA → GQA → MLA) and three precisions (FP16 → FP8 → NVFP4), built as twelve single-variable kernel steps.
2) A two-layer prediction model (pure roofline plus a per-CTA-corrected layer) whose predictions are pre-registered before each kernel is coded or measured. The gap between the layers is itself the finding: across the seven decode steps the per-CTA-corrected layer matched the measured limiter every time, while pure roofline predicted the wrong regime at every single-stream step (Section III).
3) The central finding: decode's per-CTA "wall" is work-starvation, not a fixed architectural limit. The right engine converts work into throughput. A warp-per-head CUDA-core MLA kernel is flat at 0.75 TFLOP/s, whereas the production tcgen05 kernel spans 2.4 → 1785 TFLOP/s scaling with batch × context. Even so, the roofs are reachable

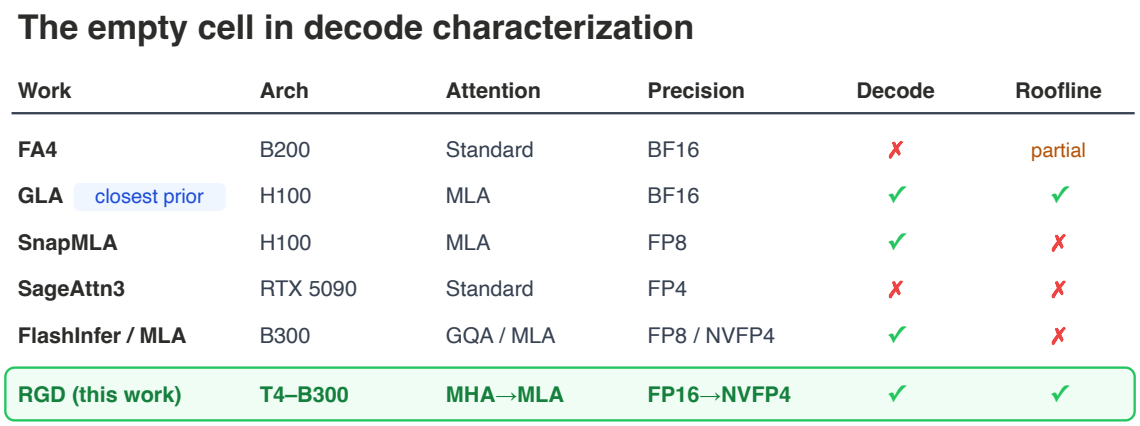

Fig. 1. The related-work landscape (works surveyed in Section VI: FA4 [6], GLA [7], SnapMLA [8], SageAttention3 [9], FlashInfer/FlashMLA [1], [2]). Existing work covers prefill, Hopper, or ships production Blackwell decode without an open roofline; only the RGD row (this work) combines sm_103 decode across MHA→GQA→MLA and FP16→FP8→NVFP4 with a prediction-vs-measured roofline. GLA is the closest prior (delta: sm_103 + KV-quant).

only at serving scale, not in common single-stream decode.

c) Honest scope: I complement, not beat, production kernels: FlashInfer, FlashMLA, and the CUTLASS reference kernel already run B300 decode and are faster by construction. The prediction-vs-measured methodology is also not novel; prior work owns it [11]. My contribution is the open application of that methodology to this workload × architecture × precision space, the per-CTA-corrected model, and the boundary results (including a negative one, Section V-C). Figure 8 states precisely what I claim and what I do not. Prediction misses are reported as first-class results throughout.

II. Background

A. Attention variants

For a sequence of length N with head dimension d, self-attention computes $O = \mathrm{softmax}(QK^\top/\sqrt{d})V$. During decode, the query is a single new token (N_q=1) attending over a KV cache of length N_k. The variants differ in how much KV must be read per generated token:

- MHA (multi-head): H_q=H_{kv}; every query head has its own KV, so the cache is $O(B \cdot H \cdot N_k \cdot d)$ and decode arithmetic intensity (AI, FLOPs per byte) is $\approx 2/b$ where b is bytes per KV element.
- GQA (grouped-query [12]): $H_{kv} < H_q$, with G=H_q/H_{kv} query heads sharing one KV group. A decode kernel can pack the G queries of a group into one CTA, reading the shared KV once and raising AI from $2/b$ to $2G/b$.
- MLA (multi-head latent, DeepSeek [13], [14]): one shared low-rank latent (dimension 576 = 512 latent + 64 RoPE) replaces all KV heads. At decode all h_q query heads attend to the single latent, so the packed row dimension is M=h_q=128 by construction, and $AI \approx 2\,h_q(2L{+}R)/((L{+}R)\,b) \approx 3.78\,h_q/b$. MLA is the only decode shape that meets the Blackwell block-scaled NVFP4 tensor-core gate ($M \geq 128$) with no speculative draft.

B. The roofline model, and why it fails for decode

The classical roofline [15] bounds attainable throughput by min(peak FLOP/s, $AI \times$ peak bandwidth); the ridge point $AI^\star$ = peak FLOP/s/peak bandwidth separates memory-bound ($AI < AI^\star$) from compute-bound ($AI > AI^\star$). Applied naively to modern GPUs, this over-predicts sustained throughput by wide margins: datasheet peaks are unreachable, and the model is blind to the L2 cache, per-CTA scheduling, occupancy, and tensor-core pipeline fill. Microbenchmark-driven analysis [11] quantifies this: naive roofline exceeds 95% error on B200 GEMMs where a microbenchmark-grounded model reaches 1.31% MAE. My per-CTA-corrected layer (Section III) adds exactly those on-chip constraints: shared-memory staging → occupancy, warp packing (GEMV vs GEMM), and pipeline depth vs available MMA tiles.

C. Blackwell Ultra (B300 / sm_103)

My headline architecture is the B300 (GB300, sm_103). I use measured constants throughout, several of which correct commonly cited datasheet values (Table I): 148 streaming multiprocessors (SMs) enabled (not 160), 8 TB/s HBM (flat versus B200; only capacity grew to 288 GB), 132.6 MB L2 (measured; the widely repeated "192 MB" appears to conflate B200's 192 GB capacity), 228 KB shared memory per SM, and a measured boost clock of 2032 MHz (below the 2600 MHz marketing figure). The tcgen05 tensor cores deliver dense peaks of 2.5 PF (FP16), 5 PF (FP8), and 15 PF (block-scaled NVFP4). I also measured the exponential throughput used by softmax: an EX2 microkernel sustains 5.33 TExp/s, only

TABLE I
Measured architectural constants (from roofline/archs.py). Blackwell values are torch device-property / microbenchmark measurements taken on the rented parts; peaks in PF are dense tensor-core datasheet figures.

	T4 sm_75	A100 sm_80	B200 sm_100	B300 sm_103
SMs	40	108	148	148
HBM BW	320 GB/s	2.0 TB/s	8 TB/s	8 TB/s
HBM capacity	16 GB	80 GB	192 GB	288 GB
L2 cache	4 MB	40 MB	132.6 MB	132.6 MB
Boost clock	1590	1410	1965	2032 MHz
smem/SM	64 KB	164 KB	228 KB	228 KB
FP16 TC peak	65 TF	312 TF	2.25 PF	2.5 PF
FP8 TC peak	–	–	4.5 PF	5 PF
NVFP4 TC peak	–	–	9 PF	15 PF

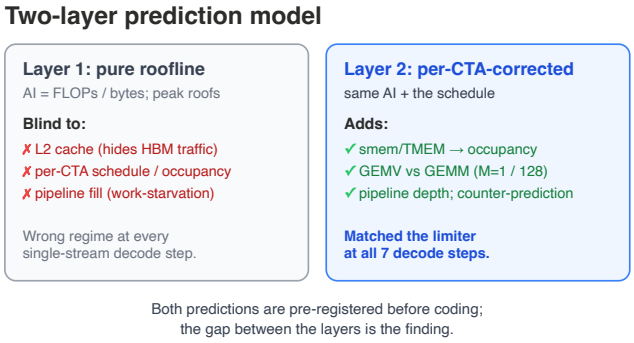

Fig. 2. The two-layer prediction model. Layer 1 (pure roofline) predicts the regime from FLOPs/bytes and peak throughputs; Layer 2 adds per-CTA structure (occupancy, GEMV-vs-GEMM packing, pipeline depth). Both are pre-registered; the gap between them is the reported result.

0.50× the 10.7 TExp/s vendor peak. I fold this datasheet gap into the model, though it is immaterial to M=1 decode (the MUFU term is $< 3\%$ of decode time).

III. Methodology

A. The two-layer prediction model

Every kernel step is accompanied by two predictions, both recorded before the kernel is written (Figure 2):

- Layer 1 (pure roofline). AI from operand traffic and FLOPs; the limiter from datasheet peak throughputs. This captures the what (how much work and how much data) but is blind to the how.
- Layer 2 (per-CTA-corrected). The same AI, plus the on-chip constraints the roofline cannot see: (a) shared-memory / tensor-memory staging → how many CTAs fit per SM (occupancy); (b) warp-level packing (M=1 GEMV vs M=128 GEMM); (c) tensor-core pipeline depth vs the number of available MMA tiles; and (d) a counter-prediction, the observation that, if made, would falsify the thesis.

The value of the exercise is that the counter-prediction makes every step publishable regardless of sign: a confirmed thesis and a confirmed counter-prediction (a negative or boundary result) are equally informative.

B. Pre-registration and single-variable steps

The twelve kernels form a chain in which each step changes exactly one variable (schedule, shape, or precision) relative to the previous, so that a measured delta is attributable (Figure 3). Predictions are committed to the repository before coding; I never retrofit a prediction to a measurement. Table II is the resulting prediction-vs-measured scorecard.

C. Measurement methodology

Vendor performance counters (ncu) are blocked on every containerized rental I used (ERR_NVGPUCTRPERM), so decode throughput is read with a counter-free proxy: effective bandwidth = KV bytes moved / measured time, reported as %HBM against peak, using CUDA-event timing. On a privileged (root) T4 I validated this proxy directly against ncu: the proxy read 13.8% HBM where ncu DRAM throughput was 12.85%, within about one percentage point (Section IV-B, v9 Task 1). Every decode reading in this paper uses the validated proxy. I additionally (i) lock clocks on the root T4 and idle-pin them on Blackwell, (ii) flush the L2 with a zeroed buffer $\geq 2\times$ L2 outside the timed window so that data genuinely comes from HBM, and (iii) push the working set past L2 (up to N_k=2,097,152, a 5× overflow of the 132.6 MB Blackwell L2) to remove the L2-residency confound. Kernels are measured on T4 (sm_75), A100 (sm_80), B200 (sm_100), and B300 (sm_103). For full transparency about compute provenance: all T4 runs used Google Colab (free tier) for JIT build, correctness, and benchmarking, except the clock-locked, L2-flushed, ncu-validated regime run (v9 Task 1), which used a root (privileged) T4 rented from vast.ai; the A100, B200, and B300 were rented from vast.ai. The privileged root T4 is the one host on which ncu counters were available; every other measurement uses the counter-free proxy above.

D. Reading the scorecard

Table II is the paper's methodological spine. Across the seven decode steps (v6–v12), the per-CTA-corrected layer (L2) predicted the measured limiter every time. Pure roofline (L1) predicted the wrong operative regime at every single-stream decode step (v6, v7, v9, v10, v11) and at v8 got the speedup magnitude right ($G\times$) while still misplacing the absolute limiter; it was vindicated only at v12, where large-batch, long-context work finally drove the production kernel toward the HBM roof it had predicted. I state this as the defensible tally (L2 correct at all seven decode steps, L1 wrong at the operative limiter for the single-stream steps) rather than any cleaner-looking ratio, because the per-step verdicts in the table are what the record supports. The prefill steps (v2–v4) contribute three further pure-roofline misses that are first-class findings in their own right (Section IV-A).

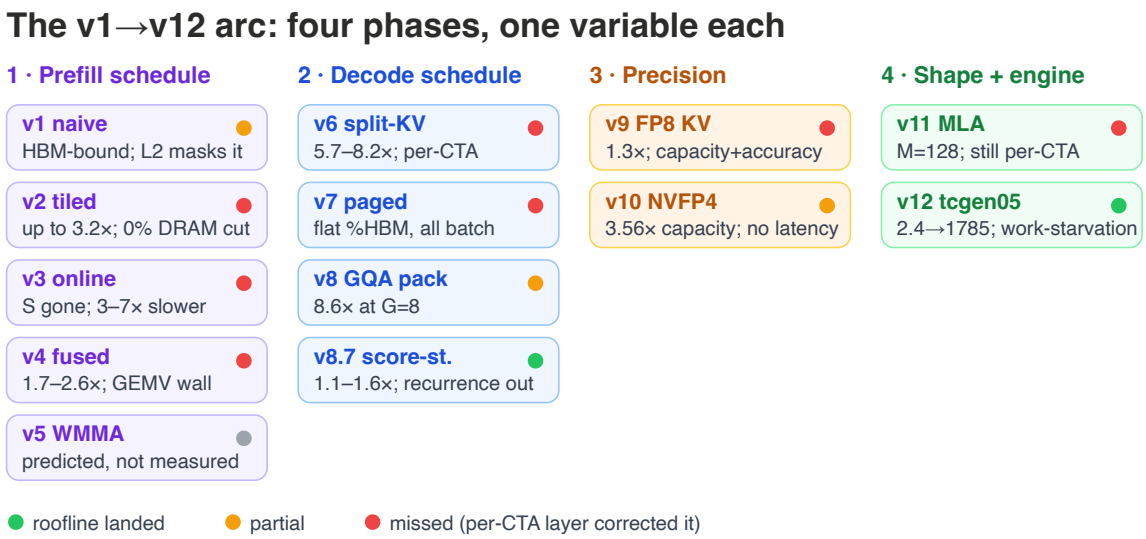

Fig. 3. The v1–v12 arc as four phases, each step changing one variable: prefill schedule (v1–v5), decode schedule (v6–v8.7), precision (v9–v10), and shape then engine (v11–v12). Dot color marks the pure-roofline verdict (green landed, amber partial, red missed and corrected by the per-CTA layer); the per-step verdicts are Table II.

TABLE II
Pre-registered prediction-vs-measured scorecard. "L1" is pure roofline; "L2" is per-CTA-corrected. An n/a in L1/L2 marks steps where only a schedule change was predicted. v8.7 is a sub-step of v8 (its inner-loop relayout); the seven main decode versions are v6–v12. The decode arc is the contribution; the per-CTA-corrected layer matches the measured limiter at every step, while pure roofline predicts the wrong regime at every single-stream decode step.

Step	Kernel (single variable)	L1 predicted	Measured limiter	L1	L2
v1	naive FP32 (baseline)	HBM, $AI \approx 0.25$	HBM, masked by L2 at mid-N	partial	n/a
v2	shared-memory tiling	HBM, ∼30× traffic cut	compute/schedule; DRAM traffic flat	miss	n/a
v3	online softmax (S off HBM)	MMA-bound	occupancy/latency; 151× off floor	miss	n/a
v4	fused single-pass (warp/row)	MMA-bound	FMA under-utilization (GEMV), 18× off	miss	n/a
v5	tensor-core WMMA	MMA, 8× lower floor	not measured (prediction only)	n/a	n/a
v6	split-KV decode	HBM, split-KV fills SMs	per-CTA (occupancy), ∼12% HBM	miss	hit
v7	paged KV + batch sweep	occupancy→bandwidth crossover	per-CTA at all batch (flat %HBM)	refuted	hit
v8	GQA M-packing (CUDA cores)	HBM, speedup $\sim G\times$	per-CTA; speedup $= G\times$, %HBM $\leq$11	split	hit
v8.7	score-stationary inner loop	remove serial recurrence	per-CTA load latency; recurrence removed	n/a	hit
v9	FP8 E4M3 KV (bytes)	capacity-only, halve floor	L2→SM load-latency win (∼1.3×)	miss	hit
v10	NVFP4 KV (bytes)	HBM, 3.55× lower floor	per-CTA; capacity 3.56×, no latency win	miss	hit
v11	MLA latent (shape, CUDA cores)	compute-flip (AI 235)	per-CTA, 0.75 TFLOP/s, 4× off cuBLAS	miss	hit
v12	tcgen05 engine (production)	HBM at scale (AI 470)	work-starved; 1–2% → 36% peak w/ $B{\times}K$	hit@scale	hit

IV. Results

A. The prefill arc (v1–v5, T4)

The first five kernels rebuild FlashAttention for prefill and establish the methodology; each pure-roofline miss teaches something the traffic model cannot see. v1 (naive, three-pass FP32) is HBM-bound in principle ($AI \approx 0.25$) but runs faster than its cache-free floor at mid-N (85.6 ms vs a 109 ms floor at N=2048, d=64) because the T4's 4 MB L2 absorbs the redundant operand reads; only at N=8192, when L2 overflows, does it converge to the floor. v2 (shared-memory tiling) is 1.3–3.2× faster than v1, but ncu shows the win is compute/scheduling, not bandwidth: DRAM traffic is essentially unchanged (1.02× at N=8192), because the L2 had already served the operands and because ∼99% of DRAM traffic is the materialized score matrix S, which is byte-identical in v1 and v2. The roofline predicted a ∼30× traffic cut; the measured cut was zero. v3 (online softmax) eliminates S from HBM entirely, dropping peak memory 125× (2164 MB → 17 MB), yet is 3–7× slower than v2, proving S was never the bottleneck on a machine that was never bandwidth-bound; the kernel sits 151× above its MMA floor, occupancy/latency-bound. v4 (fused single-pass, warp-per-row) recovers the schedule (1.7–2.6× over v2, 7.5–15× over v3) and closes the distance to the floor from 151× to ∼18×; the residual gap is FMA under-utilization, since warp-per-row scoring is GEMV-shaped, a per-key shuffle reduction dominating the two FMAs. v5 (tensor-core WMMA) is predicted to drop the floor 8× but its prefill benchmark was never executed; I report

TABLE III
v7 batch sweep at N_k=8192 (T4). %HBM is flat across a 64× batch range: no occupancy→bandwidth crossover. v7 also loses to SDPA at $B \geq 8$ (SDPA amortizes launch; v7 is already SM-saturated at B=1).

B	BH	%HBM d=64	%HBM d=128	vs SDPA d=64
1	8	9.8%	12.2%	2.65×
8	64	9.4%	11.4%	0.56×
16	128	9.4%	11.4%	0.56×
32	256	9.5%	11.6%	0.51×
64	512	10.3%	12.4%	0.50×

it as a prediction only and never as a measured result (Section V-D).

B. The decode arc (v6–v12)

The decode arc is the contribution. All decode readings use the counter-free proxy of Section III-C.

a) Split-KV and paging (v6–v7): Decode with N_q=1 leaves a single-query kernel with one CTA and idle SMs. Split-KV partitions the KV across blocks, each emitting an unnormalized partial (O, m, ℓ) that an LSE merge recombines. v6 beats the naive N_q=1 loop by 5.7–8.2× and torch SDPA by 1.5–3.3×, but reaches only 9–15% of HBM (6.5–11.6× above the bandwidth floor): the limiter is occupancy/launch, not bandwidth. v7 adds a paged block-table gather (the vLLM cache layout) and, critically, a batch sweep that refutes the predicted occupancy→bandwidth crossover: %HBM stays flat at 9.4–12.4% from BH=8 to BH=512 (Table III). Code inspection confirms the cause: shared memory caps residency at 2 blocks/SM, and at N_q=1 only one of eight warps computes, so batch merely adds waves. Decode is per-CTA-bound at every batch size, not grid-occupancy-bound, which reorders the roadmap: per-CTA efficiency (GEMV→GEMM) before bytes.

b) GQA M-packing and score-stationary (v8, v8.7): Packing the G query heads of a GQA group into the CTA's M dimension activates G warps (not one) and reads the shared KV once, raising AI from $2/b$ to $2G/b$. On plain CUDA cores this v8 Cut 1 tracks $\sim G\times$ up to G=8 (measured 8.59× at d=64, 8.71× at d=128 versus no-packing) and beats SDPA 6–10× at every batch B=1–64, reclaiming the serving regime v7 had lost. %HBM stays $\leq$ 11%: the roofline got the speedup scaling right (a first partial win in six steps) while the kernel never approaches the absolute floor. Tensor cores are the wrong tool here: a Turing WMMA variant (Cut 2) is 2.1–3.3× slower than Cut 1 even at G=16 (a full WMMA tile), pinned by the opaque-fragment staging tax, not compute. Two further single-variable ablations, double-buffered KV (v8.5) and key-ILP/occupancy (v8.6), were both null, isolating the floor as the per-row serial online-softmax recurrence. v8.7 removes it: a score-stationary relayout (lane = key) eliminates the per-key cross-lane reduction and shortens the recurrence 32×. It is the first inner-loop lever to move the clock-robust %HBM (up 2–3 points, to ∼10–12%) and beats Cut 1 by 1.1–1.6× at matched clock. This confirms the recurrence was a real, removable floor component, while a residual per-CTA ceiling remains.

c) Cross-precision KV (v9–v10): v9 stores the KV cache in FP8 E4M3 (1 byte), dequantized fused per tile. The pre-registered "capacity-only" prediction is refuted: FP8 delivers a real, clock-matched ∼1.3× decode-latency win (median across G; 0.96–1.52×), and the win shrinks as G grows (1.37× at G=2 → 0.96× at G=32, d=64). That shrinkage is the fingerprint of an L2→SM load effect that M-packing amortizes away, not an HBM effect. %HBM drops versus FP16 and never flips the limiter; per-tensor E4M3 RMSE versus FP16 KV is ∼6–7×10^{-4}. v10 stores NVFP4 (packed E2M1 nibble + per-16 micro-scale, 0.5625 B/element): measured capacity is 3.56× versus FP16 and 1.78× versus FP8. NVFP4 RMSE is $\sim2.4\times10^{-3}$, about 4× FP8's (E2M1's single mantissa bit, not a bug), and there is no latency win (%HBM falls to ∼3%). An asymmetric per-channel-K / per-token-V recipe recovers accuracy only where real (GPT-2) KV has channel-outlier structure and is layer-conditional, so I ship the robust block-16 default.

d) v9 Task 1: the confound-free per-CTA verdict: The "∼10% HBM, per-CTA-bound" reading recurs from v6, but until v9 Task 1 it was confounded: the micro-bench KV fit in L2, clocks were unlocked, and N_k never passed L2. On a root T4 I removed all three at once (clocks locked at 1590 MHz, L2 flushed, $N_k \to$ 128K so the working set is 537 MB $\gg$ 4 MB L2) with ncu finally live, the first counter-validated measurement in the project. Achieved %HBM plateaus at ∼28–29% under full occupancy (never near the ∼70% achievable ceiling), with no bandwidth knee at the L2 crossing; ncu reports L2 hit-rate 1.1% (data genuinely from HBM) yet DRAM throughput only 12.85%. The verdict is earned: decode is per-CTA / low-MLP latency-bound, not bandwidth-bound, and the counter-free proxy is validated (13.8% vs 12.85%). Bytes are not the decode wall: FP8/NVFP4 are a capacity+accuracy play, not a decode-latency play.

e) MLA decode: the shape change (v11–v12): With bytes exhausted and occupancy capped, the only lever left is the shape. v11 implements MLA on CUDA cores: all h_q=128 heads share one latent, so M=128 by construction and AI jumps to ≈235. Pure roofline calls this the arc's first compute-flip; the per-CTA-corrected layer predicts it stays per-CTA because a warp-per-head kernel packs M=128 as 16 blocks ×8 warps, not one GEMM. Measurement confirms the corrected layer on both T4 and B300: per-CTA-bound to N_k=2M past a 5× L2 overflow (%HBM ≈ 0), at 0.75 TFLOP/s (<1% of either compute peak). The headline is a ∼4× gap: torch dense-MQA, which routes M=128 into cuBLAS tensor-core GEMMs, beats my CUDA-core kernel ∼4× on B300. This is a clean result either way: the MLA shape is correct and tensor-core-friendly, but the CUDA-core default is the wrong

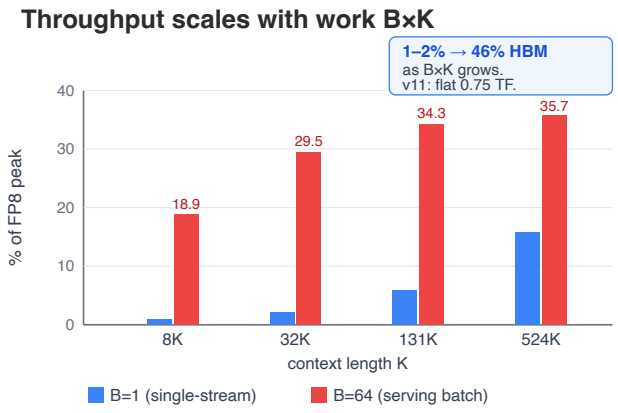

Fig. 4. v12 (production tcgen05 MLA decode, B300): FP8 throughput versus total work $B \times K$. Single-stream decode (B=1, moderate K) sits at 1–2% of the 5 PF FP8 peak; throughput climbs to 35.7% peak / 46.3% HBM as $B \times K$ grows. The floor is work-starvation, not a fixed wall.

tool. (I attribute the gap to cuBLAS exploiting the tensor cores; the torch baseline itself was not profiled, so the "M=128-as-one-TC-GEMM" benefit is shown structurally and externally [7], not by a torch-side measurement.) Capacity is 202× versus MHA / 12.6× versus GQA-8, and the latent absorption identity (absorbed kernel ≡ explicit per-head materialization) is verified in the correctness suite.

v12 changes only the engine. On the B300 box I found that CUTLASS 4.6 example 77 already ships the production tcgen05 MLA decode kernel (Sm100FmhaMlaKernelTmaWarpspecialized: M=128 by static_assert, FP8-native, paged + split-KV + LSE), so v12's deliverable became characterizing that canonical kernel directly rather than reinventing NVIDIA's scheduler. Figure 4 and Table IV give the result: at realistic single-request decode (B=1, $K \leq$32K) even the production kernel runs at 1–2% of FP8 peak, exactly the pre-registered per-CTA/pipeline-depth floor; but throughput scales with total work $B \times K$ to 1785 TFLOP/s = 35.7% of FP8 peak / 46.3% of HBM at B=64, K=524,288. FP16 tracks FP8 (not compute-bound), and –verbose confirms 174 KB shared memory →∼1 CTA/SM (the measured capacity pressure). The transition is smooth in $B \times K$ with no hard knee.

This corrects the hedged v6–v11 reading. "Per-CTA-bound forever" was a small-$B \times K$ artifact of the CUDA-core kernels: v11's warp-per-head GEMV is flat at 0.75 TFLOP/s, whereas the same-shape tcgen05 kernel spans 2.4 → 1785 TFLOP/s (∼70× v11 at B=1/K=8K, ∼1000× at K=524K). The per-CTA wall is work-starvation; the right engine converts work into throughput (Figure 5). I measured NVIDIA's canonical kernel; I complement, not beat it. (Honest deviation: the pre-registered v12 row assumed NVFP4 storage, AI 835; ex77

TABLE IV
v12 FP8 decode throughput vs work (B300, CUTLASS ex77, sm_103a). Peaks: FP8 5 PF, HBM 8 TB/s.

B	K	TFLOP/s	% FP8 peak	% HBM
1	256	2.4	0.05%	0.15%
1	8192	51.4	1.0%	1.4%
1	32768	112.5	2.2%	2.9%
1	524288	790.6	15.8%	20.5%
64	8192	944.8	18.9%	25.5%
64	131072	1717.3	34.3%	44.5%
64	524288	1785.3	35.7%	46.3%

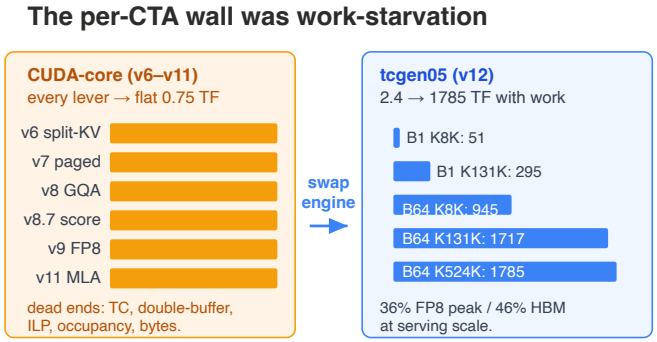

Fig. 5. The correction. v6–v11 CUDA-core decode is flat near the per-CTA floor (v11 MLA: 0.75 TFLOP/s regardless of $B \times K$). The tcgen05 engine (v12) spans 2.4 → 1785 TFLOP/s as work grows, so "per-CTA-bound forever" was a small-$B \times K$ artifact; the wall is work-starvation and the engine is the lever.

is FP8-storage, AI 470 < the FP8 ridge 625, so pure roofline predicts HBM-bound, and at scale the kernel indeed leans to the HBM roof, 46.3% HBM above its 35.7% compute fraction. NVFP4-storage and the native-FP4 compute arm remain unmeasured; see Section V-C.)

C. The cross-architecture per-CTA ceiling

The strongest evidence that the decode floor is a shape property, not a hardware limit, is architecture-independence. At B=1 decode the CUDA-core kernels achieve ∼40 GB/s on T4, B200, and B300 (Table V): 11–14% of the T4's 320 GB/s but only ∼0.5% of Blackwell's 8 TB/s. If decode were bandwidth-bound the percentage would be constant; instead the absolute ∼40 GB/s ceiling holds across a 25× span of peak bandwidth, which is the per-CTA/low-MLP latency signature. On B300 the CUDA-core MLA kernel confirms this to N_k=2M (a 5× L2 overflow), with %HBM flat near zero throughout.

V. Discussion

A. The engine-ridge framework

Once the compute engine is a free variable, each precision has its own ridge (Figure 6). On B300 the ridge is $AI^\star$ = peak/8 TB/s: ≈312 for FP16 (2.5 PF), 625 for FP8 (5 PF), and 1875 for NVFP4 (15 PF). This

TABLE V
The architecture-independent ∼40 GB/s per-CTA ceiling at B=1 decode. The absolute achieved bandwidth, not the percentage, is invariant across a 25× span of peak HBM.

Arch	HBM peak	achieved @ B=1	% of peak
T4 (sm_75)	320 GB/s	∼36–46 GB/s	11–14%
B200 (sm_100)	8 TB/s	∼40 GB/s	∼0.5%
B300 (sm_103)	8 TB/s	∼40 GB/s	∼0.5%

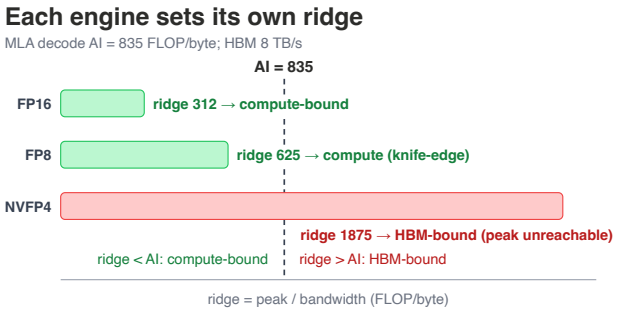

Fig. 6. Engine-ridge comparison on B300. Each precision's ridge scales with its tensor-core peak (FP16 ≈312, FP8 625, NVFP4 1875 FLOP/byte). Decode arithmetic intensities land left of every ridge, so NVFP4's 15 PF peak overshoots into the HBM-bound region and is unreachable for decode.

reframes the precision levers. FP8 MLA-latent storage gives $AI \approx 470 < 625$, so even with the tensor-core engine the kernel is HBM-leaning, consistent with v12's measured 46.3% HBM > 35.7% compute at scale. NVFP4 pushes AI higher but its 15 PF peak lifts the ridge to 1875, so the FP4 compute peak is doubly unreachable for decode: the workload is HBM-bound before it is compute-bound. The engine-ridge picture is why a bigger FLOP peak does not, by itself, help a memory-lean decode workload.

B. Implications for serving systems

The measurements separate two regimes cleanly. Single-stream, moderate-context decode, the common interactive case, leaves even the production kernel at 1–2% of peak; the lever there is not precision or algorithm but work (batching and context length that fill the tensor-core pipeline). Long-context, large-batch serving does reach the roofs (46% HBM at B=64, K=524K). Consequently the value of FP8/NVFP4 KV is capacity and accuracy (halving or quartering the cache, letting more sequences or longer contexts fit), not decode latency: the FP8 latency win is L2-resident-only and reverses under an L2 flush, and NVFP4 is latency-negative past L2 (12–30% slower than FP16). Practitioners should treat low-precision KV as a capacity knob and reach for engine plus batch to convert decode work into throughput.

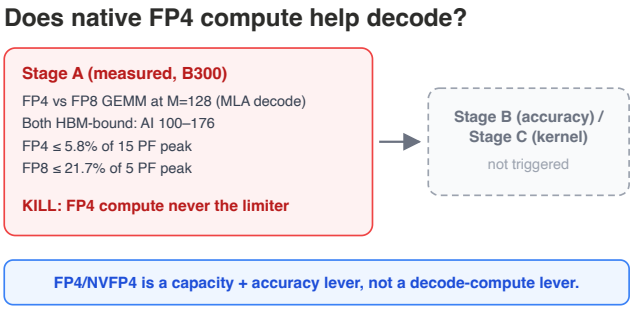

Fig. 7. The native-FP4-compute boundary test (Section V-C). At the M=128 MLA-decode QK-GEMM shape, both FP4 and FP8 GEMMs are HBM-bound at every context length; FP4 tops out at 5.8% of its 15 PF peak. The result is a pre-registered negative: FP4/NVFP4 stays a capacity+accuracy lever, not a decode-compute lever.

C. Does native FP4 compute help MLA decode? (Boundary test)

Because MLA decode meets the block-scaled NVFP4 tensor-core gate ($M \geq 128$) by construction, it is tempting to expect that native FP4 compute (15 PF, 3× FP8) would accelerate it. I pre-registered the prediction "no" and tested it directly on B300 (sm_103a, CUDA 12.9, CUTLASS v4.5.2; a separate measurement campaign on a different rented box than v12's, hence the different CUTLASS version): a sweep of the M=128 MLA-decode QK-GEMM over $K \in \{256, 512, 576\}$, $N \in \{1\text{K} \ldots 524\text{K}\}$, comparing FP4 (CUTLASS example 72a, block-scaled NVFP4→bf16) against FP8 (cuBLAS E4M3 via torch._scaled_mm, matched bf16 output). Figure 7 and the data of record confirm the kill: both precisions are HBM-bound at every N (AI 100–176 ≪ the ridges 1875 / 625), reaching at most 5.8% of the 15 PF FP4 peak and 21.7% of the 5 PF FP8 peak. Native FP4 compute is therefore never the operative limit, and the M=128 packing advantage is a red herring for decode. The raw FP4/FP8 ratio flips with N: FP4 leads 1.4–2.7× at small N (both tiny, latency-bound), while FP8 wins at large N, the realistic long-context regime (e.g. K=576, N=131072: FP8 1086 vs FP4 739 TFLOP/s). Neither direction is a compute win; the gap is a bandwidth / operand-size effect, i.e. the same capacity/storage lever as Sections IV-B and V-B, not tensor-core throughput.

This is a cross-harness comparison (a CUTLASS example FP4 kernel versus a tuned cuBLAS FP8 kernel), so the head-to-head ratio is not clean; but the kill rests on the kernel-quality-independent peak-fraction and roofline (AI ≪ ridge caps FP4 far below its compute peak regardless of tuning), not on the ratio. The consequence is that accuracy follow-ups for a fused FP4-compute kernel are not triggered, and FP4/NVFP4 remains a capacity+accuracy lever. (I use "Stage A" for this GEMM-compute test; it is distinct from the v10 asymmetric-precision accuracy ablation, which I do not abbreviate

the same way.)

D. Threats to validity

I disclose the following. Counter-free proxy: the %HBM proxy is ncu-validated only on a root T4 (13.8% vs 12.85%); on B300 ncu was blocked (unprivileged container) and the proxy carries as a throughput estimate, not a counter measurement. Clock variability: Colab throttles; the load-bearing verdicts use locked-clock root-T4 runs and idle-pinned Blackwell runs, but cross-kernel μs/tok on Blackwell is not fully clock-controlled. Single-seed accuracy: FP8/NVFP4 RMSE figures are single-seed on gpt2-small and on synthetic KV; they are my substrate numbers, not published constants, and the $\sim$3.5$\times$ FP4-vs-FP8 latent gap should be read as such. v5 was never benchmarked: its 8$\times$ floor drop is a prediction only; I never present it as measured. nsys provenance: the v10 B300 schedule figures come from an off-notebook nsys 2025.3.2 run (the committed cell ran 2025.1.3, which records an empty sm_103 trace); the v11 schedule was re-captured with 2025.3.2 and is non-empty in-repo. No SOTA comparators run: FlashMLA-class kernels are orders of magnitude faster by construction; I make no wall-clock claim against them. The contribution is the open methodology and characterization.

VI. Related Work

Figure 1 places this work in the landscape; I discuss the three closest points and then briefly survey the rest.

a) The closest prior: Grouped-Latent Attention (GLA) [7] is the nearest work: an open, roofline-documented study of MLA-style latent-attention decode, showing on H100 that latent attention sits at the roofline knee and can run up to 2$\times$ faster than standard attention. It is the template for my decode roofline; my delta is exactly sm_103 (Blackwell Ultra) plus KV-quantization (FP8/NVFP4) and the per-CTA-corrected two-layer model. Attn-QAT [10] pushes attention to 4-bit with quantization-aware training on Blackwell, but for the prefill/training forward-and-backward path; my delta is decode, MLA, and the roofline characterization. Microbenchmark-driven analytical modeling [11] establishes why naive roofline fails on modern GPUs (1.31% MAE on B200 GEMMs vs >95% for naive roofline) and owns the prediction-vs-measured method; I cite it as prior art and contribute the application to attention decode.

b) Attention kernels and MLA: The FlashAttention lineage [3]–[5] established the fused, memory-efficient prefill kernel, and FlashAttention-4 [6] co-designs the algorithm and pipeline for Blackwell (BF16 prefill, up to 1605 TFLOP/s / 71% utilization on B200), but it targets B200 prefill and is not characterized on sm_103 decode. MLA originates with DeepSeek-V2/V3 [13], [14]; FlashMLA [2] is the production Hopper MLA decode kernel (memory-bound 3000 GB/s, compute-bound 580 TFLOP/s on H800), and FlashInfer [1] is a customizable attention engine deployed in vLLM and SGLang. These ship Blackwell decode but publish no open roofline.

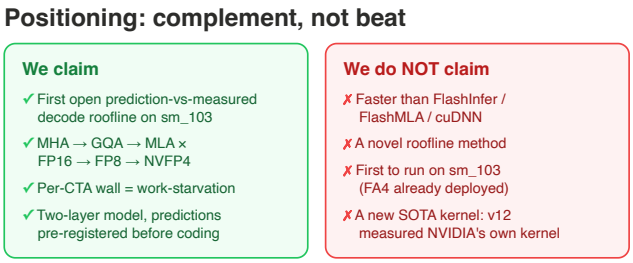

Fig. 8. Paper positioning: what I claim (an open, prediction-vs-measured decode characterization on sm_103; the two-layer model; the work-starvation correction; the FP4-compute boundary) and what I do not (a SOTA wall-clock win over production kernels).

c) Low-precision attention and MLA: SnapMLA [8] builds an FP8 MLA decode pipeline with RoPE-aware per-token KV quantization on Hopper; SageAttention3 [9] accelerates attention with microscaling FP4 tensor cores on Blackwell consumer parts (prefill). Hardware-centric analyses of DeepSeek's MLA [16] and of the Blackwell microarchitecture [17] characterize the platform but not the prediction-vs-measured decode roofline across attention types and precisions that I target.

VII. Conclusion

I presented an open, prediction-vs-measured roofline characterization of attention decode on B300 / sm_103, built as twelve single-variable kernel steps spanning MHA $\rightarrow$ GQA $\rightarrow$ MLA and FP16 $\rightarrow$ FP8 $\rightarrow$ NVFP4. The central finding is that decode's apparent per-CTA ceiling is work-starvation: a warp-per-head CUDA-core MLA kernel is flat at 0.75 TFLOP/s, but the production tcgen05 kernel on the same shape spans 2.4 $\rightarrow$ 1785 TFLOP/s as batch $\times$ context grows. "Per-CTA-bound forever" was therefore a small-work artifact, and the right engine converts work into throughput, though only in the high-throughput serving regime. The per-CTA-corrected layer of my two-layer model matched the measured limiter at every decode step, while pure roofline predicted the wrong regime at every single-stream step and was vindicated only at serving scale. A pre-registered boundary test settles that native NVFP4 compute does not accelerate MLA decode (both FP4 and FP8 GEMMs HBM-bound, $\leq$5.8% / 21.7% of peak), so FP4/NVFP4 is a capacity+accuracy lever, not a decode-compute one. Throughout I complement, not beat, the production kernels; the contribution is the open characterization, the model, and the boundary results. The empty cell is filled. Figure 8 summarizes the scope.

a) Future work: Native FP4 compute is settled negative for decode, so the open directions are: grouped-latent / sparse attention (GLA, DSA/CSA) as a different shape; a cross-generation roofline spine (a clean A100 point

to show the ∼40 GB/s per-CTA ceiling is architecture-independent across four generations); and ncu validation of the %HBM proxy on a privileged B300 to upgrade the counter-free readings on sm_103.

Artifact Availability

The twelve kernels, roofline tool, benchmark harness, and the notebooks of record for every measurement are open source at https://github.com/gkienpham-cmd/flashattention-cuda.

Acknowledgment

I developed the codebase for this study (the twelve CUDA kernels, the roofline tool, and the benchmark harness) in pair-programming sessions with Anthropic's Claude models, Claude Opus 4.8 and Claude Fable 5, via Claude Code. I pre-registered all predictions and directed and verified all measurements and conclusions myself.

References

[1] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "Flash-Infer: Efficient and customizable attention engine for LLM inference serving," in Proceedings of Machine Learning and Systems (MLSys), 2025, arXiv:2501.01005.

[2] DeepSeek-AI, "FlashMLA: Efficient multi-head latent attention kernels," https://github.com/deepseek-ai/FlashMLA, 2025, production MLA decode kernel for Hopper; 3000 GB/s memory-bound, 580 TFLOP/s compute-bound (H800).

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in Advances in Neural Information Processing Systems (NeurIPS), 2022, arXiv:2205.14135.

[4] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," arXiv preprint arXiv:2307.08691, 2023.

[5] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," arXiv preprint arXiv:2407.08608, 2024.

[6] T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, "FlashAttention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling," arXiv preprint arXiv:2603.05451, 2026, blackwell B200 BF16 prefill; up to 1605 TFLOP/s (71% utilization).

[7] T. Zadouri, H. Strauss, and T. Dao, "Hardware-efficient attention for fast decoding," arXiv preprint arXiv:2505.21487, 2025, introduces Grouped-Tied Attention (GTA) and Grouped-Latent Attention (GLA); open MLA-decode roofline on H100.

[8] Y. Zhang, Z. Su, S. Hu, R. Yang, W. Wu, Y. Qian, Y. Xie, and X. Cai, "SnapMLA: Efficient long-context MLA decoding via hardware-aware FP8 quantized pipelining," arXiv preprint arXiv:2602.10718, 2026.

[9] J. Zhang, J. Wei, P. Zhang, X. Xu, H. Huang, H. Wang, K. Jiang, J. Zhu, and J. Chen, "SageAttention3: Microscaling FP4 attention for inference and an exploration of 8-bit training," arXiv preprint arXiv:2505.11594, 2025.

[10] P. Zhang, M. Noto, W. Tan, C. Jiang, W. Lin, W. Zhou, and H. Zhang, "Attn-QAT: 4-bit attention with quantization-aware training," arXiv preprint arXiv:2603.00040, 2026.

[11] A. Jarmusch and S. Chandrasekaran, "Microbenchmark-driven analytical performance modeling across modern gpu architectures," arXiv preprint arXiv:2605.04178, 2026, 1.31% MAE on B200 GEMMs vs >95% for naive roofline.

[12] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," arXiv preprint arXiv:2305.13245, 2023.

[13] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," arXiv preprint arXiv:2405.04434, 2024, introduces Multi-head Latent Attention (MLA).

[14] ——, "DeepSeek-V3 technical report," arXiv preprint arXiv:2412.19437, 2024.

[15] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," Communications of the ACM, vol. 52, no. 4, pp. 65–76, 2009.

[16] R. Geens and M. Verhelst, "Hardware-centric analysis of DeepSeek's multi-head latent attention," arXiv preprint arXiv:2506.02523, 2025.

[17] A. Jarmusch and S. Chandrasekaran, "Microbenchmarking NVIDIA's blackwell architecture: An in-depth architectural analysis," arXiv preprint arXiv:2512.02189, 2025.